



Powersa B2B Projesi: İleri Düzey Derinlemesine Sistem Analizi

Bu belge, Powersa B2B projesine tam anlamıyla hakim olmanız için hazırlanmış "aşırı detaylı" mimari ve kod okuma rehberidir. Sıradan bir genel bakışın ötesine geçerek, **hangi dosya veritabanının neresiyle konuşur, sipariş süreci nasıl işler, Logo'dan ürün nasıl çekilir** gibi konulara tek tek değiniyoruz.



1. Büyük Resim: Sistem Nasıl Çalışıyor?

Proje 3 temel bacadan oluşur:

1. **Logo ERP (Ana Kasa):** Şirketin tüm fatura, müşteri, ürün ve muhasebe kayıtlarının tutulduğu MS SQL Server tabanlı yerel program.
2. **Powersa B2B Backend (Laravel):** İnternet üzerinde koştan ve Logo'nun aynası olan hızlı, güvenli veritabanı (PostgreSQL) ve API katmanı.
3. **Powersa B2B Frontend (Next.js):** Müşterilerin, plasiyerlerin ve depo personelinin gördüğü süper hızlı, modern web sitesi.



2. BACKEND API (Laravel 12) - apps/api

Klasik bir Laravel uygulamasının çok ötesinde, büyük bir "Enterprise" mimarisi kurulmuş.

A) Modeller (Veritabanı Tabloları ve İlişkileri)

apps/api/app/Models altındaki dosyalar sistemin damarlarıdır. İşte görevleri:

• Ticaret ve Sepet Modelleri:

- Cart & CartItem: Kullanıcıların sepetleri. Veritabanında saklanır (tarayıcı kapanınca kaybolmaz).
- Order & OrderItem: Onaylanmış siparişler ve satırları.
- OrderStatusHistory: "Sipariş alındı", "Hazırlanıyor", "Kargoya verildi" gibi tüm statü değişimlerinin loglarını tutar.

• Ürün ve Stok Modelleri:

- Product: Ana ürün kartı (Logo'daki ITEMS tablosunun karşılığı).
- Brand, Category: Ürün marka ve kategorileri.
- StockMovement, StockSummary: Ürünün hangi depoda ne kadar olduğu ve anlık stok değişimleri.
- Warehouse: Depo tanımları (Örn: Merkez Depo, Şube 1 Deposu).

- BasePrice, PriceList: Fiyat listeleri ve ürüne ait taban fiyatlar.
- ProductCodeAlias: Ürünün farklı barkodları veya fabrika/üretici (OEM) kodları.
- **Finans ve Tahsilat Modelleri:**
 - LedgerEntry: Müşterinin cari ekstresindeki tek bir satır (Bakiye, Borç, Alacak).
 - Collection: Tahsilat makbuzu/kaydı.
- **Fiziksel Mağaza / POS Modelleri:**
 - PosSale & PosSaleItem: Mağazadaki dokunmatik ekrandan yapılan peşin veya kredi kartlı anlık satış.
 - PosSession: Kasiyerin gün başlangıcında kasayı açması ve akşam "Gün Sonu" yapıp kapatması (Z Raporu).
 - PosExpense: Kasadan yapılan nakit giderler (Örn: Çay, kargo ücreti).
 - PosPayment: Pos satışının ödeme kırılımı (Örn: 100TL Nakit, 50TL Kredi Kartı).
- **Saha ve Depo Modelleri:**
 - Shipment & ShipmentItem: Depodan çıkan kargo/sevkiyat işlemleri.
 - ShipmentScan: Depo görevlisinin el terminaliyle tek tek okuttuğu barkodların kaydı.
 - Vehicle & VehicleProduct: Sevkiyat araçları ve araç içi stok takibi (Araçta satış yapan plasiyerler için).
- **İade Modelleri:**
 - ReturnRequest: Müşterinin veya plasiyerin başlattığı iade süreci.
 - PurchaseReceipt: (Mal Kabul) Tedarikçiden veya şubeden gelen malın depoya giriş evrakı.

B) Rotalar (apps/api/routes/api.php)

Sistemin dışarıya açılan kapılarıdır. Rotalara bakıldığında yetki gruplarına (middleware: role, menu) göre ekranların kilitlendiğini görüyoruz.

1. **/auth:** Login işlemleri. (Sanctum ile Cookie yaratılır).
2. **/pos:** Kasiyer yetkisine (role:cashier,point) sahip olanların girebildiği rotalar. Gün açılış-kapanış, satış ve masraf kayıtları atılır.

3. **/warehouse:** Depo yetkisine (role:warehouse) sahip kişilerin kullandığı, "Siparişleri Topla", "Barkod Okut", "Kargo Etiketleri Yazdır" işlemlerinin rotaları.
4. **/reports:** Müşteri bakiyeleri, ciro raporları, tahsilat raporları. Sadece adminler ve yetkili plasiyerler girebilir.
5. **/integrations/logo/:** Frontend buraya istek ATMAZ! Bu rotalara sadece tools/logo-sync aracı arka plandan bağlanır. Logo'dan gelen güncel ürünleri sisteme kaydeder veya B2B'den alınan siparişleri Logo'ya yazdırmak için bekleyenleri çeker (/integrations/logo/orders/pending).

3. FRONTEND WEB (Next.js) - apps/web

Müşteri paneli (B2B) ve Admin paneli ayrımı yoktur. Giriş yapan kişinin rolüne göre sol menü şekillenir.

A) Dosya ve Klasör Hiyerarşisi

apps/web/src/ içindeki en önemli parçalar:

- **app/(app)/:** Ana arayüzdür. Üstte header, solda sidebar vardır.
 - **dashboard/:** Giriş yapınca çıkan özet ekranı.
 - **orders/:** Sipariş listesi ve detayları.
 - **products/:** Katalog, ürün arama, stok/fiyat görme.
- **app/(auth)/:** Boş, beyaz bir sayfada sadece giriş kutusunun olduğu klasör.
- **app/(print)/:** İçinde sidebar (yan menü) BARINDIRMAYAN, faturayı/kargo etiketini yazıcıdan (Ctrl+P) çıkartmak için özel tasarlanmış siyah-beyaz rotalar.

B) Bileşenler (Components - apps/web/src/components)

Tüm sayfalar aslında bu bileşenlerin yap-boz gibi birleştirilmesinden oluşur.

- **cart/:** Sepet sağdan açılan çekmece (Drawer) mi yoksa tam sayfa mı? Tüm sepet mimarisi burada.
- **pos/:** Fiziksel bir mağaza kasası görünümünü sağlayan özel bileşenler. Ürünü okut, listeye düşsün, ödeme al arayüzleri.
- **warehouse/:** El terminalleri (telefon/tablet) için tasarlanmış büyük butonlu, barkod tarayıcı destekli okutma ekranları.
- **mal-kabul/:** Depoya mal alırken kullanılan ekranlar.
- **customer-card/:** Yeni cari kart (müşteri) açma talebi formları.

- **reports/**: Grid (Tablo) yapıları, excel'e dışa aktar butonları, finansal grafikler.
- **ui/**: Radix UI altyapısı ile kurulmuş buton, input, modal (dialog), dropdown (select) gibi çekirdek bileşenler. Burası hiç değişmez, sadece diğer klasörler tarafından çağırılır.

C) Veri Yönetimi

- **lib/api.ts**: Tüm backend çağrıları (Axios) buradan geçer. Çöken sunucuya karşı yeniden deneme veya çerezleri yetkilendirme (credentials: include) işlemleri burada ayarlıdır.
- **Zustand (hooks/useCart.ts, hooks/useUser.ts)**: Sepette kaç ürün var, şu an hangi müşterinin adına işlem (impersonation) yapıyoruz gibi bilgiler RAM'de tutulur ve sayfa değişse de sıfırlanmaz.

⚙️ 4. LOGO ERP ENTEGRATÖRÜ - tools/logo-sync

Projenin en teknik ve hataya yer bırakmayan yeridir. **Node.js** ortamında çalışır.

A) Çalışma Mantığı

Logo, MS SQL Server kullanır. Bu yazılım bir Windows Makine üzerinde "Görev Zamanlayıcı (Task Scheduler)" sayesinde sürekli çalışır.

1. **Dışa Aktarım (Export)**: Web sitesinde bir sipariş oluşturulunca, logo-documents-export.mjs scripti çalışır. B2B veritabanındaki (PostgreSQL) siparişi okur.
2. **Dönüşüm (SQL İşleme)**: sql/powersa-b2b-order-shipment-pos-write-procedure.sql içindeki binlerce satırlık özel prosedürü kullanarak, bu siparişi Logo'nun diline çevirir ve Logo'nun ORFICHE (Sipariş Fişi), ORFLINE (Sipariş Satırı) tablolarına ekler.
3. **İçe Aktarım (Sync)**: logo-products-sync.mjs çalışır. Logo'daki ITEMS tablosuna bakar. Yeni eklenen ürün varsa veya fiyatı değişen varsa, bunu JSON paketine çevirip Laravel'in /integrations/logo/products/sync adresine fırlatır.

B) Nöbetçi Sistem (logo-sync-daemon.mjs)

Sürekli çalışan arka plan programıdır. Ürün aktarımı gibi çok ağır işleri gece yarısına saklayabilir, sipariş ve ödeme gibi saniyelik işleri ise anında yakalayıp içeri/dışarı aktarır.

C) Karmaşık Kurallar (logo-product-target-selection.mjs)

Hangi ürünlerin B2B'de görüneceğini belirler. Örneğin Logo'da 10.000 ürün olabilir ama sadece "WEB_SATIS=1" olanları veya "B2B_KATEGORI"si dolu olanları B2B veritabanına aktarmak için filtreleme kurallarını barındırır.

🔒 5. Özel Geliştirici Kuralları (Neden Çok Dikkatliyiz?)

AGENTS.md kod standartlarını belirler:

- **SPA Auth Zorunluluğu:** Next.js ve Laravel farklı teknolojiler olmasına rağmen aynı domain (veya sub-domain) altında çalışır. Laravel Sanctum, Next.js'ten gelen isteğin aynı ajanstan gelip gelmediğini anlamak için /sanctum/csrf-cookie adresine boş bir istek attırır. Next.js tarayıcıya "XSRF-TOKEN" adında bir çerez yerleştirir ve tüm işlemler bu çerez ile güvene alınır. JWT (JSON Web Token) **kullanılmaz**, çünkü çerezler çok daha güvenlidir.
- **Local Geliştirme (Macbook):** İki farklı Mac üzerinde çalışıldığı için .env (şifre) dosyaları Git'e yollanmaz. Localde .env.example kopyalanıp oluşturulur.
- **Dar Kapsam:** Bu devasa sistemde bir hata yaparsak, örneğin sipariş onayı yanlış giderse muhasebe kayıtları (Logo) bozulur. Bu yüzden ben kod yazarken SADECE benden istediğiniz özelliği kapsayan dosyaları değiştireceğim, çalışan düzene dokunmayacağım.

ÖZETLE

Bu sistem:

1. **Frontend**'de modern ve inanılmaz hızlı bir arayüz sunan (React/Next.js)
2. **Backend**'de çok katmanlı, güvenli ve genişleyebilir bir mimari sunan (Laravel 12)
3. **Lokal ağda** Logo Muhasebe programının SQL sunucusuna sızıp saniyeler içinde B2B ile konuşmasını sağlayan (Node.js)

bir sanat eseridir. İstediğiniz tüm detayları listeledim. Soru sormaya hazırız, istediğiniz dosyayı düzenlememi, veya nasıl çalıştığını anlatmamı isteyebilirsiniz!

Powersa B2B Teknik Eğitim Kılavuzu: Teknolojiler ve Veritabanı

Kral, bu belgeyi senin için projenin röntgeni olarak hazırladım. Projede "Ne kullanılmış?" ve "Veritabanında hangi tablo ne işe yarıyor?" sorularının mutlak cevabını burada bulacaksın.

Bölüm 1: Kullanılan Yazılım Dilleri ve Teknolojiler

Proje 3 temel ayağa oturtulmuş ve her biri için endüstri standardı diller seçilmiş.

1. Backend (Arka Plan & İş Mantığı)

- **Dil: PHP 8.2+** (Nesne Yönelimli, kesin tipli - strictly typed).
- **Framework: Laravel 12** (PHP dünyasının en modern, en büyük framework'ü).
- **Veritabanı: PostgreSQL** (Çok güçlü, ilişkisel SQL veritabanı).
- **Kimlik Doğrulama: Laravel Sanctum** (Token yerine Cookie tabanlı, çalınması imkansız yakın güvenlik sistemi).
- **Arama Motoru: Meilisearch** (Ürün arama çubuğuna "Demir" yazdığında milisaniyede sonucu getiren, RAM üzerinde çalışan arama motoru).
- **Kuyruk ve Önbellek: Redis ve RabbitMQ (AMQP)** (Arkada Logo'ya veri atarken sistemi kilitlememek için kullanılır).

2. Frontend (Ön Yüz & Arayüz)

- **Dil: TypeScript / JavaScript** (Tip güvenli JS).
- **Kütüphane: React 19** (Dünyanın en popüler UI kütüphanesi).
- **Framework: Next.js 16 (App Router)** (Sayfaların SEO uyumlu olmasını ve çok hızlı yüklenmesini sağlayan React framework'ü).
- **Stil (CSS): Tailwind CSS v4** (Satır içi class'larla çok hızlı stil yazmayı sağlar, CSS dosyası yazma derdini bitirir).
- **UI Bileşenleri: shadcn/ui & Radix UI** (Hazır, mükemmel tasarımlı buton, menü ve popup'lar).
- **Veri Çekme (Fetching): TanStack React Query v5** (Ekranı "Yükleniyor..." ikonlarını otomatik yapar, çekilen veriyi önbelleğe alır).
- **Durum Yönetimi (State): Zustand** (Sepetteki ürün sayısını hafızada tutan çok hafif yapı).

3. Entegrasyon Aracı (Logo Sync)

- **Dil: Node.js** (JavaScript'in sunucu tarafında çalışan hali).
- **Veritabanı Bağlantısı: MSSQL (mssql paketi)** (Logo'nun çalıştığı Microsoft SQL Server'a direkt bağlanmak için).
- **Zamanlama: Windows PowerShell (.ps1)** ve .cmd dosyaları (Node.js kodlarını Windows'un içine bir hizmet/servis olarak gömmek için).

Bölüm 2: Veritabanı Tabloları (PostgreSQL Şeması)

Laravel tarafında (PostgreSQL) tam 30'dan fazla tablo var. Hangi tablo neyi saklar?

Kullanıcı ve Yetki Tabloları

- **users:** Sisteme giriş yapan herkes (Müşteri, Kasiyer, Plasiyer, Admin). E-posta, şifre ve Logo'daki eşlenik ID'si tutulur.
- **roles & role_user:** Kimin ne yetkisi var? (Örn: id: 1, name: admin, id: 2, name: cashier). Bir user birden çok role sahip olabilir.
- **user_notes:** Personellerin sisteme veya birbirlerine bıraktıkları küçük notlar.

B2B ve Müşteri Tabloları

- **customers:** Logo'daki Cari Kartların kopyasıdır. Firmanın adı, vergi dairesi, Logo'daki LogicalRef'i burada tutulur.
- **dealers:** Ana bayiler. Bir müşteri bir bayiye bağlı olabilir.
- **customer_card_requests & customer_card_request_attachments:** Sisteme kayıt olmak isteyen yeni müşterilerin başvuru formları ve yükledikleri vergi levhası/imza sirküleri.

Ürün ve Katalog Tabloları

- **products:** Logo'daki ITEMS tablosu. Ürünün adı, Logo ID'si, birimi, KDV oranı vb. tutulur. Fiyatı burada tutulmaz!
- **brands & categories:** Ürün markaları ve B2B kategori ağacı.
- **product_code_aliases:** Bir ürünün birden fazla barkodu veya fabrika kodu (OEM) varsa onlar buraya yazılır. Arama yaparken bu tablodan da aranır.
- **hot_products:** "Haftanın Fırsatları" veya "Öne Çıkan Ürünler" vitrinini oluşturan tablo.
- **base_prices & price_lists:** (Dinamik fiyatlama) Ürünlerin liste fiyatları. B2B, müşteri girdiğinde onun iskontosuna göre bu tablodan fiyat hesaplar.

- stock_summary & stock_movements: Hangi depoda hangi üründen kaç tane var (stock_summary). Ve bu stok ne zaman eklendi/çıktı (stock_movements).

Sipariş ve Sepet Tabloları

- carts & cart_items: Kullanıcı sepete ürün atıp siteden çıkarsa kaybolmasın diye sepettekiler veritabanına yazılır.
- orders & order_items: Onaylanmış siparişler ve içindeki ürün kalemleri. Logo'ya başarıyla aktarılsa logo_export_status kolonu güncellenir.
- order_status_histories: "Sipariş saat 14:00'da kargoya verildi", "15:00'da teslim edildi" gibi zaman çizelgelerini (timeline) tutar.

Finans ve Muhasebe Tabloları

- ledger_entries: "Cari Hesap Ekstresi". Müşterinin Logo'daki faturaları (borç) ve ödemeleri (alacak) satır satır buraya aktarılır.
- collections: Sistem üzerinden veya havale/EFT ile yapılan tahsilat makbuzları.

Fiziksel Mağaza (POS) Tabloları

- pos_sessions: Kasiyerin mesaisi. (Sabah 08:00 Kasa açıldı, akşam 18:00 Kasa kapandı).
- pos_sales & pos_sale_items: Tıpkı bakkaldaki yazar kasa gibi, barkodla okutulup hızlıca satılan perakende ürünler.
- pos_payments: O satışın nasıl ödendiği (Örn: 50 TL Kredi Kartı, 10 TL Nakit).
- pos_expenses: Kasadan manav için alınan 10 TL gibi masraf çıkışları.

Depo ve Lojistik Tabloları

- warehouse_shipments & warehouse_shipment_items: Bir sipariş kargoya verilirken toplanan koliler.
- warehouse_shipment_scans: Depo elemanının elindeki tabancayla ürünlerin barkodunu okutarak pakete atma logları (Yanlış ürün gitmesini engeller).
- vehicles & vehicle_products: Sahada dolaşan kamyonlar (Mobil araç depoları) ve içlerindeki anlık stoklar.
- purchase_receipts & purchase_receipt_items: Mal Kabul! Dışarıdan veya başka şubeden depoya gelen ürünlerin giriş fişleri.
- return_requests: Müşterinin veya plasiyerin "Bu ürün bozuk" diyerek iade başlattığı süreç.

⚙ Sistem ve Entegrasyon Tabloları

- integration_sync_events & integration_sync_states: Logo ile B2B'nin haberleşme geçmişi. "Dün gece 03:00'da ürünler çekilirken hata oldu" gibi loglar buraya yazılır.
- report_runs: Büyük excel raporları oluşturulurken kullanıcıyı bekletmemek için arka planda çalışan rapor motorunun durum tabloları.
- jobs: Laravel'in arka plan görev (Queue) kuyruğu. "Logo'ya sipariş gönder" emri önce bu tabloya düşer, arka plandaki işçi (Worker) sırası gelince bunu işler.

Özetle Ne Görüyoruz?

Bu veritabanı şeması, sıradan bir web sitesi şeması değildir. **Mikro bir ERP'dir.** Kendi içinde deposu, irsaliyesi, pos sistemi, finans ekstresi ve kullanıcı yetkileri olan, Logo'nun modern bir "Web Yansıması" olarak kodlanmıştır.

Görüp de *"Vay canına, bunu nasıl kurgulamışlar"* dediğin veya *"Ben yeni bir tablo eklemek istiyorum, nereye ekleyeceğiz?"* diyeceğin yeri söyle, orayı inceleyelim!



Powersa B2B: Projenin Gizli Sırları ve Arka Plan Mekanizmaları

"Bana sormadıklarımı da söyle" dedin. Haklısın, profesyonel bir yazılım sadece klasörlerden ve tablolardan ibaret değildir. Projenin .env (şifre ve ayar) dosyalarının derinliklerine inerek, kodların arasına saklanmış altyapısal sırları (Infrastructure & Ops) senin için çıkardım.

İşte projeye tamamen hakim olmak için bilmen gereken "Sahne Arkası" dinamikler:

1. 🚀 Işık Hızında Arama: Meilisearch

Klasik e-ticaret sitelerinde kullanıcı arama çubuğuna "Çivi" yazdığında veritabanında LIKE %Çivi% şeklinde arama yapılır ve bu sistemi yavaşlatır. Bu projede **Meilisearch** adında devasa ve aşırı hızlı bir açık kaynak arama motoru kullanılıyor.

- **Nasıl Çalışır?** Laravel tarafında app/Observers/ProductObserver.php adında bir dinleyici (Observer) var. Logo'dan web'e yeni bir ürün geldiğinde veya fiyatı değiştiğinde, Laravel bu Observer'ı tetikler. Observer hemen arka planda bir SyncProductToMeiliJob (Kuyruk Görevi) başlatır ve ürünü Meilisearch sunucusuna yollar. Müşteri web sitesinde arama yaparken PostgreSQL veritabanına DEĞİL, direkt bu RAM üzerinde çalışan Meilisearch sunucusuna bağlanır. (İlgili ayarlar .env dosyasındaki MEILISEARCH_HOST kısmındadır).

2. ☁ Bulut Depolama: AWS S3 Kullanımı

Sistem sadece kendi sunucusuna dosya kaydetmekle kalmıyor, Amazon'un bulut dosya deposu olan **AWS S3**'e entegre.

- **Ne İşe Yarıyor?** Dosyaları (Örn: Müşteri cari kartı açarken yüklenen vergi levhası belgeleri, CustomerCardRequestAttachment tablosundaki dosyalar veya binlerce ürün görseli) sunucuyu şişirmemek için AWS S3'e (veya ona uyumlu bir servise) yükleme altyapısına sahip. (.env dosyasındaki AWS_ACCESS_KEY_ID ve AWS_BUCKET ayarları).

3. 🚦 Arka Plan İşçileri (Queue & Jobs - Redis/RabbitMQ)

Kullanıcı "Siparişi Tamamla" butonuna bastığında ekran donup kalmasın diye "Kuyruk" (Queue) sistemi kurulmuş.

- Bir kullanıcıya mail atılacaksa, Excel raporu oluşturulacaksa (Reports jobları) veya ürün Meilisearch'e gönderilecekse, bu işlem **Redis** kuyruğuna atılıyor.

- Ayrıca daha sağlam bir mesaj kuyruğu servisi olan **RabbitMQ** entegrasyonu da hazırda bekliyor (LOGO_WRITE_RABBITMQ_HOST). İleride çok yüksek hacimli (saniyede yüzlerce) fatura aktarımı gerekirse sistem altyapı olarak buna hazır.
-

4. 🕵️ Hata Yakalama Radarı: Laravel Telescope

Laravel projelerinin en efsane aracı olan **Telescope** aktif edilmiş (TELESCOPE_ENABLED=true).

- **Ne Yapar?** Sisteme gelen her bir isteği, çalıştırılan her bir SQL sorgusunu (kaç milisaniye sürdüğüne kadar), gönderilen tüm mailleri ve alınan tüm hataları kaydeder. Sistemde bir hata olursa log dosyalarına saatlerce bakmak yerine <http://.../telescope> (muhtemel adresine) girerek hatanın satırını bile görebileceğin devasa bir geliştirici aracı projenin içinde hazır duruyor.
-

5. 🏢 Fiziksel Şubeler: Şube Bazlı Kasa (Cashbox) Yönetimi

Bu sadece bir B2B değil, aynı zamanda şubeleri olan bir B2B. .env yapılandırmasına dikkatlice bakıldığında:

- POS_ERZURUM_POINT_CASHBOX_CODE
 - POS_BATUM_POINT_CASHBOX_CODE Gibi şube spesifik kasa (Cashbox) kodları var. Yani Erzurum'daki görevli sisteme girdiğinde Logo'da yaptığı işlemler direkt "Erzurum Kasası"na yazılıyor, Batum'daki girdiğinde Batum'a. Sistem çok-lokasyonlu (Multi-branch) bir perakende altyapısına göre dizayn edilmiş.
-

6. 🐳 Sunucu ve Yayına Alma Altyapısı (Docker & Hostinger)

- **Lokal Geliştirme:** Proje kök dizininde docker-compose.yml bulunuyor. Bu şu demek: Bilgisayarına tek tek veritabanı, redis vs. kurmana gerek yok. Docker yüklüyse tek komutla tüm sunucu altyapısı Mac'inde çalışmaya başlar.
 - **Canlı Sunucu (Hostinger):** Projede infra/hostinger adında gizli bir klasör var. Bu klasörün içinde uygulamanın Hostinger paylaşımlı veya VPS sunucularında nasıl çalıştırılacağı, ayarlarının nasıl yapılacağına dair kurulum scriptleri ve notları tutuluyor. (Deploy ederken çok işe yarar).
-

Mükemmel Resmi Çizdik

Şu ana kadar sana:

1. **Ana Mimarının Ne Olduğunu (Klasörler ve Dosyalar)**
2. **Hangi Dillerle Neden Yazıldığını ve Veritabanını (PostgreSQL Şeması)**
3. **Projenin Sahne Arkasındaki Altyapısını (S3, Redis, Meilisearch, Telescope)**

Ayrıntısıyla çıkardım. Senden benden bir satır bile saklı hiçbir şey kalmadı!

Artık direksiyondasın. Hangi kodu değiştireyim, hangi sayfayı açıp okuyayım, veya nereye yeni bir özellik ekleyeyim? Emrin olur.

⚙️ Mekaniğin Kalbi: logo-sync-daemon.mjs (Nöbetçi)

Sistemde manuel bir "Senkronize Et" butonu yoktur. Logo'nun bulunduğu Windows Server'da arka planda sürekli çalışan tools/logo-sync/logo-sync-daemon.mjs isimli bir Node.js servisi (daemon) vardır.

Bu daemon bir sonsuz döngü (while (!stopping)) içinde çalışır ve görevleri ikiye ayırır:

1. **Hızlı Görevler (Fast Steps):** Birkaç saniyede bir çalışır. Müşteri (Cari) güncellemeleri, bakiye güncellemeleri, alınan siparişleri veya Pos satışlarını Logo'ya aktarma.
2. **Yavaş Görevler (Slow Steps):** Birkaç dakikada (veya saatte) bir çalışır. Örn: 10.000 ürünün güncel stok miktarını Logo'dan çekmek.

🔄 Senaryo 1: Web'den Bir Sipariş Geldi, Logo'ya Nasıl Düşer? (İhracat/Export Süreci)

Müşteri B2B sitesine girdi, sepete 5 ürün attı ve "Siparişi Onayla" dedi. Laravel bu siparişi veritabanına status: pending_export olarak kaydeder. Ardından mekanizma şöyle işler:

1. **Daemon Harekete Geçer:** Nöbetçi daemon, hızlı görevler listesindeki documents-export adımına gelir ve tools/logo-sync/logo-documents-export.mjs dosyasını çalıştırır.
2. **B2B'ye "Bekleyen Var mı?" Sorusu:** Script, Laravel API'sine (apps/api/routes/api.php içindeki GET /integrations/logo/orders/pending rotasına) bir HTTP isteği atar.
3. **Laravel Cevap Verir:** LogoOrderExportController@index çalışır ve veritabanındaki henüz Logo'ya aktarılmamış siparişleri JSON formatında Node.js'e verir.
4. **Logo SQL'e Doğrudan Müdahale:** Script, dönen her siparişi alır. mssql paketini kullanarak doğrudan Logo'nun veritabanına bağlanır. tools/logo-sync/sql/powersa-b2b-order-shipment-pos-write-procedure.sql içindeki devasa **Stored Procedure'ü** tetikler.
 - Bu prosedür çok karmaşıktır. Neden? Çünkü Logo'da bir sipariş açmak için sadece "Şu üründen 5 tane" demek yetmez. Fişin türü, cari kodun bulunması, ürünün KDV'si, satır indirimi, ambar (depo) numarası gibi onlarca değişkeni Logo'nun ORFICHE ve STLINE tablolarına kusursuzca yazar.
5. **Geri Bildirim (Acknowledge):** Eğer SQL prosedürü başarıyla çalışıp Logo'da fişi oluşturursa, Logo'nun verdiği Fiş Numarasını alır. Script hemen Laravel'e geri döner (POST /integrations/logo/orders/ack) ve *"104 numaralı siparişi Logo'ya*

yazdım, Logo'daki fiş numarası da 000000000015, bunu sisteminde Onaylandı olarak işaretler" der.

6. **Laravel Kaydı Kapatır:** Sipariş B2B'de "Logo'ya Aktarıldı" statüsüne geçer.

Senaryo 2: Logo'da Bir Ürünün Fiyatı veya Stoku Değişti, B2B Nasıl Anlar? (İthalat/Sync Süreci)

Muhasebede biri Logo programını açtı ve "Demir Çubuk" ürününün fiyatını 100 TL'den 120 TL'ye güncelledi.

1. **Daemon Harekete Geçer:** Nöbetçi daemon bu sefer tools/logo-sync/logo-products-sync.mjs scriptini çalıştırır.
2. **Logo Veritabanını Tarama:** Script Logo veritabanına bağlanır (LG_XXX_ITEMS, LG_XXX_PRCLIST, LV_XXX_GNTOTST tablolarına). En önemli nokta şudur: **Sadece değişenleri (Lookback) bulur.** Tüm veritabanını çekmez. "Son 1 saatte ITEMS tablosunda güncellenenleri getir" tarzı bir SQL Select atar.
3. **Veriyi Paketleme:** Bulduğu "Demir Çubuk" ürününü, güncel KDV'si, OEM kodu, birim setleri (Adet/Koli) ve yeni fiyatı ile beraber bir JSON nesnesine çevirir.
4. **Laravel'e Fırlatma:** POST /integrations/logo/products/sync adresine bu JSON'u yollar.
5. **Laravel'in İşlemesi:** LogoProductSyncController bu isteği alır. Veritabanındaki Product modelini bulur, fiyatını günceller.
6. **Meilisearch Tetiklenmesi:** Fiyat güncellendiği an Laravel tarafındaki ProductObserver devreye girer ve "Arama motorundaki (Meilisearch) fiyatı da güncelle ki, müşteri arama çubuğuna Demir yazdığında eski fiyatı görmesin" diyerek arama motoru indeksini de saniyesinde eşitler.

Güvenlik ve Mimari Detaylar

- **Peki İki Sistem Birbirine Nasıl Güveniyor?** Logo Server'daki Node.js scripti .env içindeki POWERSA_SYNC_KEY (Özel şifre) isimli bir anahtar ile Bearer Token veya özel bir header üreterek Laravel'e istek atar. Laravel LogoIntegrationMiddleware ile bu anahtarı kontrol eder, doğru değilse kapıdan kovar.
- **Ya Sunucu Çökerse?** Örneğin Node.js Logo'ya siparişi yazdı ama o saniye internet koptu ve Laravel'e "Yazdım" (Ack) diyemedi. Bir sonraki döngüde Laravel o siparişi "Hala bekliyor" diye tekrar gönderir. Node.js SQL prosedürü içeride "Ben

bu B2B ID'li siparişı zaten işlemiştim!" diyerek çift (mükerrer) kayıt açılmasını engeller (Buna yazılımda **Idempotency** denir, muazzam bir kurgudur).

Kısacası mimari: Logo'daki olayları "çekip" B2B'ye iten (Product Sync) ve B2B'de oluşanları "çekip" Logo'ya iten (Document Export), merkezinde Node.js + SQL Prosedürlerinin olduğu bir **Kuyruk (Queue) ve Zamanlayıcı (Polling)** mekanizmasıdır.

Aynen öyle kral, sistemin en kritik parçası o "Köprü Yazılımı" dediğimiz kısım. Bizim projede bunun yeri **tools/logo-sync** klasörünün ta kendisidir.

Karşı bilgisayarda (Logo'nun kurulu olduğu ana Windows sunucusunda veya bilgisayarda) çalışan kodlar, işte bu klasörün içindeki .mjs (Node.js) ve .sql (SQL Prosedürleri) dosyalarıdır.

Karşı Bilgisayardaki Kurulum Aslında Nedir?

Karşı bilgisayarda çok sihirli bir şey yok, yapılan işlem temelde şudur:

1. Sizin sunucunuzda bir klasör açılır (Örneğin: C:\PowersaB2B).
2. Bizim projemizdeki tools/logo-sync klasörünün içindeki her şey oraya kopyalanır.
3. O bilgisayara **Node.js** kurulur.
4. Klasörün içine .env dosyası konulur ve Logo'nun SQL şifreleri yazılır.
5. Son olarak logo-sync-daemon.mjs dosyası Windows'un "Görev Zamanlayıcı (Task Scheduler)" sistemine eklenir ki bilgisayar her açıldığında bu köprü yazılımı arka planda sessizce çalışmaya başlasın.

Köprü Yazılımında (Karşı Bilgisayarda) Bir Değişiklik Yapmak İstersen Süreç Nasıl İşler?

Diyelim ki "Logo'dan ürünlerin ağırlık (KG) bilgisini de çekelim" diye bir kural eklemek istedik. Süreç şu 4 adımla ilerler:

1. **Önce Kendi Mac'imizde Yaparız (Local Geliştirme):** Sen bana söylersin, ben senin bilgisayarındaki tools/logo-sync klasörü içindeki (örneğin logo-products-sync.mjs ve ilgili SQL dosyalarında) gerekli kod değişikliklerini yazarım.
2. **Değişiklikleri GitHub'a Kaydederiz:** Kodlarımızı yazar, terminalden git commit ve git push yaparak projenin ana deposuna (GitHub'a) göndeririz ki kaybolmasın, temiz kalsın.
3. **Karşı Bilgisayara Aktarım (Deploy):** Karşı bilgisayara Uzak Masaüstü (RDP), AnyDesk veya TeamViewer ile bağlanılır. O bilgisayardaki C:\PowersaB2B klasörüne gidilir. Eğer o bilgisayarda Git kuruluysa sadece git pull yazarak bizim yeni kodlarımızı oraya çekeriz. Git kurulu değilse, senin Mac'indeki değişen dosyaları (örneğin güncellediğim logo-products-sync.mjs dosyasını) kopyalayıp o bilgisayardaki klasörün içine yapıştırıp eski dosyayla değiştirirsin. (Genelde sadece değişen dosyaları sürükle-bırak yapmak en pratiğidir).
4. **Yeniden Başlatma (Restart):** Karşı bilgisayarda kodlar değiştiği için, eski hafızayla çalışan arka plan servisini (daemon) durdurup yeniden başlatmak gerekir. Windows Görev Yöneticisinden veya Görev Zamanlayıcıdan program

durdurulup tekrar açılır ve köprü yazılımı artık bizim yeni yazdığımız kurallara (Örn: Ağırlık bilgisini de çekme) göre çalışmaya başlar!

Yani kısacası; **kodları senin bilgisayarında (tools/logo-sync içinde) yazıp geliştireyoruz, bitince o dosyaları alıp karşı bilgisayardaki klasöre kopyalıyoruz.**

1. Tam Otomatik Modu Yeniden Başlatmak (Önerilen)

Arka planda sessizce bekleyen ve her şeyi sırası gelince (Siparişler, Ürünler, Bakiyeler) kendi kendine kontrol eden asıl nöbetçi servisi başlatmak için: 🖱️ **run-sync-daemon.cmd** dosyasına çift tıklaman yeterlidir.

(Not: Bu dosya çalışırken aynı anda iki tane açılmasın diye .sync-daemon.lock adında bir kilit klasörü oluşturur ve işlem bitince siler. Arka planda siyah bir ekran (CMD) açılır veya tamamen arka plan hizmeti olarak çalışmaya devam eder, logları da sync-daemon.log dosyasına yazar).

2. Sadece Tek Bir İş Manuel Olarak Tetiklemek İstersen

Bazen "Sistemin otomatik taramasını beklemeyeyim, şu an sadece ürünlerin fiyatları güncellendi onu hızlıca web sitesine atsın" diyebilirsin. Bu durumda arka plan programını çalıştırmadan, sadece nokta atışı iş yapan scriptlere çift tıklayabilirsin:

- **run-sync-products.cmd:** Sadece Logo'daki ürünleri, stokları ve fiyatları alıp B2B sitesine gönderir. (İşlem bitince ekran kapanır).
- **run-sync-customers.cmd:** Sadece müşteri (Cari) bilgilerini günceller.
- **run-sync-documents-export.cmd:** B2B sitesinde bekleyen siparişleri veya Pos satışlarını alıp hemen Logo'ya fatura/fiş olarak yazar.
- **run-sync-ledger.cmd:** Sadece müşterilerin Logo'daki cari bakiye (borç/alacak) dökümlerini eşitler.

Özetle;

Eğer karşı bilgisayarda işlemi tamamen durdurduysan ve sistemin normal, otomatik rutinde (kendi kendine her şeyi tarayarak) çalışmasına devam etmesini istiyorsan klasörün içindeki **run-sync-daemon.cmd** dosyasına çift tıklayarak sistemi tekrar nöbete dkebilirsin.

B2B Tam Olarak Nedir?

B2B (Business to Business): "Şirketten Şirkete" ticaret demektir. Bizim günlük hayatta Trendyol'dan veya Hepsiburada'dan ayakkabı almamız **B2C (Business to Consumer - Şirketten Tüketicieye)**'dir. B2C sitelerinde herkes aynı fiyatı görür, kredi kartıyla alır geçer.

Ancak **B2B'de durum farklıdır:**

- Müşteriler sıradan vatandaş değil, senin bayilerin veya kurumsal müşterilerindir.
- Her bayinin sana özel bir **İskonto Oranı** (İndirimi) vardır. A bayisi ürünü 100 TL'ye görürken, B bayisi 80 TL'ye görebilir.
- Kredi kartıyla ödeme yapabildikleri gibi, "Açık Hesap (Cari)" yani veresiye de alabilirler.
- Siparişleri adetle değil, genelde koliyle veya paletle verirler.
- İade talepleri, cari hesap (borç/alacak) ekstreleri çok daha karmaşıktır. İşte bizim **Powersa B2B** projemiz, senin bayilerinin (veya plasiyerlerinin) girip sana özel fiyatlarla toptan sipariş verdiği, borcunu gördüğü, hesabını kapattığı kurumsal bir portaldır.

Bilmen Gereken "Yabancı" Teknik Terimler Sözlüğü

Yazılım dünyasında ve bu projede sürekli duyacağın kelimeler şunlardır:

1. ERP (Enterprise Resource Planning)

- **Okunuşu:** İ-Ar-Pi
- **Anlamı:** Kurumsal Kaynak Planlaması.
- **Bizdeki Karşılığı:** Sizin kullandığınız **Logo** yazılımı bir ERP'dir. Şirketin muhasebesini, deposunu, faturasını, personelini tek bir yerden yöneten dev programlara ERP denir. Bizim B2B sitemiz ERP ile (yani Logo ile) konuşur.

2. API (Application Programming Interface)

- **Okunuşu:** Ey-Pi-Ay
- **Anlamı:** Uygulama Programlama Arayüzü.
- **Bizdeki Karşılığı:** İki farklı yazılımın birbiriyle konuşmasını sağlayan garson veya elçidir. Örneğin web sitesi (Frontend) veritabanından ürünleri isterken doğrudan veritabanına bağlanmaz, Backend'deki **API'ye** "Bana ürünleri ver" der, API de veritabanından alıp ona sunar.

3. Endpoint (Uç Nokta)

- **Anlamı:** API'nin siparişi aldığı masa numarası gibidir.
- **Bizdeki Karşılığı:** Örneğin `http://site.com/api/orders` bir Endpoint'tir. Web sitesi buraya istek atarsa API siparişleri getirmesi gerektiğini anlar.

4. JSON (JavaScript Object Notation)

- **Okunuşu:** Cey-sın
- **Anlamı:** Yazılımların birbiriyle konuşurken kullandığı standart "kutu" veya "mesajlaşma dili".
- **Bizdeki Karşılığı:** Logo'daki bir ürün veritabanından alınıp B2B'ye gönderilirken düz metin olarak değil, şöyle bir JSON formatında gönderilir:

json

```
{ "urun_adi": "Demir", "fiyat": 150, "stok": 50 }
```

İki bilgisayar (Node.js ve Laravel) birbirini bu format sayesinde anlar.

5. Daemon veya Background Service

- **Okunuşu:** Di-mın
- **Anlamı:** Arka planda sessizce çalışan hayalet program. Ekranda penceresi yoktur.
- **Bizdeki Karşılığı:** Sizin Logo sunucusuna kurduğumuz o köprü yazılımı (logo-sync-daemon). Sen bilgisayarı kullanırken o arkada görünmez şekilde sürekli "Yeni sipariş var mı? Stok değişmiş mi?" diye kontrol eder.

6. Sync (Senkronizasyon)

- **Okunuşu:** Sink
- **Anlamı:** İki tarafı eşitlemek.
- **Bizdeki Karşılığı:** Logo'da fiyat 100 TL iken, B2B'de 80 TL kalmışsa sistem "Sync" (Senkronize) olur ve B2B'yi de 100 TL yapar.

7. Middleware (Ara Katman)

- **Okunuşu:** Midıl-veyr
- **Anlamı:** Kapıdaki güvenlik görevlisi.
- **Bizdeki Karşılığı:** Bir kullanıcı "Siparişler" sayfasına girmek (veya o Endpoint'e istek atmak) istediğinde, istek önce Middleware'e takılır. Middleware bakar: *"Bu adamın Kasiyer yetkisi var mı? Yok. O zaman içeri giremez, geri dön."* der.

8. Deployment (Deploy Etmek)

- **Okunuşu:** Di-ploy-mint
- **Anlamı:** Yazılan kodu "Canlıya Almak" (Yayına sokmak).
- **Bizdeki Karşılığı:** Ben kodları yazdığım da sadece bizim bilgisayarlarımızda çalışır. Müşterilerin görebilmesi için bu kodları asıl sunucuya (Hostinger'a vb.) gönderme ve sistemi yeniden başlatma işlemine Deploy denir.

9. Repository veya Kısaca "Repo"

- **Okunuşu:** Ri-po
- **Anlamı:** Projenin tüm dosyalarının, kodlarının ve geçmişinin tutulduğu depo.
- **Bizdeki Karşılığı:** Bizim kodlarımızı tuttuğumuz GitHub üzerindeki powersab2b sayfası bir Repo'dur.

10. CSRF (Cross-Site Request Forgery)

- **Okunuşu:** Si-es-ar-ef
- **Anlamı:** Hackerların başka siteler üzerinden senin sitene sahte istek atmasını sağlayan bir saldırı türü.
- **Bizdeki Karşılığı:** Bizim sistemimiz Laravel Sanctum denen bir sistemle CSRF'yi tamamen engeller. Arayüz (Frontend) login olmadan önce backend'den gizli bir "Token (Jeton)" alır ve her işlemde o jetonu gösterir. Bu sayede sahte istekler reddedilir.